

この章では、拡張関数についての理解を深め、スコープ関数を使ってよりイディオムに沿ったコードを書く方法を学びます。

Scope functions

スコープ関数

プログラミングにおいて、スコープとは、変数やオブジェクトが認識される範囲のことです。最もよく言及されるスコープは、グローバルスコープとローカルスコープです：

- **グローバルスコープ** – プログラム内のどこからでもアクセスできる変数またはオブジェクト。
- **ローカルスコープ** – 定義されたブロックまたは関数内でのみアクセス可能な変数またはオブジェクト。

Kotlin には、オブジェクトの周囲に一時的なスコープを作成し、そこでコードを実行できるスコープ関数もあります。

スコープ関数を使えば、一時的なスコープ内ではオブジェクト名を参照する必要がなくなるため、コードをより簡潔にすることができます。スコープの機能に応じて、キーワード `this` を使ってオブジェクトを参照するか、キーワード `it` を使って引数として指定することで、そのオブジェクトにアクセスできます。

Kotlin には、`let`、`apply`、`run`、`also`、`with` の計 5 つのスコープ関数があります。

Each scope function takes a lambda expression and returns either the object or the result of the lambda expression. このツアーでは、各スコープ関数の機能と使い方を解説します。

You can also watch the [Back to the Stdlib: Making the Most of Kotlin's Standard Library](#) talk on scope functions by Sebastian Aigner, Kotlin developer advocate.

また、Kotlinの開発者アドボケイト（支持者）であるSebastian Aigner氏による、スコープ関数に関する講演「[Back to the Stdlib: Making the Most of Kotlin's Standard Library](#)」もご覧いただけます。

{style="tip"}

Let

コード内でnullチェックを行い、その後、返されたオブジェクトを使ってさらなる処理を実行したい場合は、`let`スコープ関数を使用してください。

次の例を考えてみましょう：

```
fun sendNotification(recipientAddress: String): String {
    println("Yo $recipientAddress!")
    return "Notification sent!"
}

fun getNextAddress(): String {
    return "sebastian@jetbrains.com"
}

fun main() {
```

```
    val address: String? = getNextAddress()
    sendNotification(address)
}
```

```
{validate = "false"}
```

この例には2つの関数があります：

- `sendNotification()`：関数パラメータ `recipientAddress` を受け取り、文字列を返す。
- `getNextAddress()`：引数を持たず、文字列を返す関数です。

この例では、Null許容の `String` 型を持つ変数 `address` を宣言しています。しかし、`sendNotification()` 関数を呼び出すと問題が発生します。この関数は、`address` が `null` になる可能性を想定していないからです。その結果、コンパイラはエラーを報告します：

```
Argument type mismatch: actual type is 'String?', but 'String' was expected.
引数の型が一致しません：実際の型は「String?」ですが、期待されていた型は「String」です。
```

初心者向けツアーで学んだ通り、if 条件式を使って null チェックを行うか、[エルビス演算子 `?:`](#) を使用することができます。しかし、返されたオブジェクトをコードの後半で利用したい場合はどうすればよいでしょうか？ これは、if条件 **および** else分岐を使用することで実現できます：

```
fun sendNotification(recipientAddress: String): String {
    println("Yo $recipientAddress!")
    return "Notification sent!"
}

fun getNextAddress(): String {
    return "sebastian@jetbrains.com"
}

fun main() {
    //sampleStart
    val address: String? = getNextAddress()
    val confirm = if(address != null) {
        sendNotification(address)
    } else { null }
    //sampleEnd
}
```

```
{kotlin-runnable="true" id="kotlin-tour-scope-function-let-non-null-if"}
```

ただし、より簡潔な方法は、`let` スコープ関数を使用することです：

```
fun sendNotification(recipientAddress: String): String {
    println("Yo $recipientAddress!")
    return "Notification sent!"
}
```

```
}

fun getNextAddress(): String {
    return "sebastian@jetbrains.com"
}

fun main() {
    //sampleStart
    val address: String? = getNextAddress()
    val confirm = address?.let {
        sendNotification(it)
    }
    //sampleEnd
}
```

{kotlin-runnable="true" id="kotlin-tour-scope-function-let-non-null"}

この例では：

- `address` および `confirm` という名前の変数を宣言します。
- `address` 変数に対して、`let` スコープ関数用のセーフコールを使用します。
- `let` スコープの関数内に一時的なスコープを作成します。
- `sendNotification()` 関数をラムダ式として `let` スコープの関数に渡します。
- 一時スコープを使用して、`it` を通じて `address` 変数を参照しています。
- 結果を `confirm` 変数に代入します。

この方法を用いれば、コード内で `address` 変数が `null` 値になる可能性に対処でき、後でコード内で `confirm` 変数を使用できるようになります。

Apply

`apply` スコープ関数を使用すると、コードの後の段階ではなく、生成時にクラスインスタンスなどのオブジェクトを初期化できます。このアプローチにより、コードの可読性と管理しやすさが向上します。

Consider the example:

```
class Client() {
    var token: String? = null
    fun connect() = println("connected!")
    fun authenticate() = println("authenticated!")
    fun getData() : String {
        println("getting data!")
        return "Mock data"
    }
}

val client = Client()

fun main() {
    client.token = "asdf"
    client.connect()
```

```
// connected!
client.authenticate()
// authenticated!
client.getData()
// getting data!
}
```

{kotlin-runnable="true" id="kotlin-tour-scope-function-apply-before"}

この例には、`token` というプロパティを1つ持ち、`connect()`、`authenticate()`、`getData()` の3つのメンバ関数を持つ `Client` クラスが含まれています。

この例では、`main()` 関数内で、`Client` クラスのインスタンスとして `client` を作成し、その `token` プロパティを初期化した後、メンバ関数を呼び出しています。

この例は簡潔ですが、実際の現場では、クラスインスタンス（およびそのメンバ関数）を作成してから、設定して使用できるようになるまでには、しばらく時間がかかることがあります。ただし、`apply` スコープ関数を使用すれば、コード内の同じ場所で、クラスのインスタンスに対するメンバ関数の作成、設定、および使用をすべて行うことができます：

```
class Client() {
    var token: String? = null
    fun connect() = println("connected!")
    fun authenticate() = println("authenticated!")
    fun getData() : String {
        println("getting data!")
        return "Mock data"
    }
}

//sampleStart
val client = Client().apply {
    token = "asdf"
    connect()
    // connected!
    authenticate()
    // authenticated!
}

fun main() {
    val result: String = client.getData()
    // getting data!
}

//sampleEnd
```

{kotlin-runnable="true" id="kotlin-tour-scope-function-apply-after"}

この例では:

- `Client` クラスのインスタンスとして `client` を作成します。
- `client` インスタンスに対して `apply` スコープ関数を適用します。

- `apply` スコープ関数内に一時的なスコープを作成するため、`client` インスタンスのプロパティや関数にアクセスする際に、明示的に `client` を参照する必要がなくなります。
- `apply` スコープ関数にラムダ式を渡して、`token` プロパティを更新し、`connect()` および `authenticate()` 関数を呼び出します。
- `main()` 関数内で、`client` インスタンスの `getData()` メンバ関数を呼び出します。

ご覧の通り、この手法は、大規模なコードを扱う際に便利です。

Run

`apply`と同様に、`run`スコープ関数を使ってオブジェクトを初期化することもできますが、コード内の特定の時点でオブジェクトを初期化し、**かつ**即座に結果を計算したい場合には、`run`を使用する方が適しています。

前回の `apply` 関数の例を続けてみましょう。ただし今回は、`connect()` 関数と `authenticate()` 関数をグループ化し、すべてのリクエストで呼び出されるようにします。

For example:

```
class Client() {
    var token: String? = null
    fun connect() = println("connected!")
    fun authenticate() = println("authenticated!")
    fun getData() : String {
        println("getting data!")
        return "Mock data"
    }
}

//sampleStart
val client: Client = Client().apply {
    token = "asdf"
}

fun main() {
    val result: String = client.run {
        connect()
        // connected!
        authenticate()
        // authenticated!
        getData()
        // getting data!
    }
}

//sampleEnd
```

{kotlin-runnable="true" id="kotlin-tour-scope-function-run"}

この例では:

- `Client` クラスのインスタンスとして `client` を作成します。
- `client` インスタンスに対して `apply` スコープ関数を適用します。

- `apply` スコープ関数内に一時的なスコープを作成するため、`client` インスタンスのプロパティや関数にアクセスする際に、明示的に `client` を参照する必要がなくなります。
- `token` プロパティを更新するラムダ式を、`apply` スコープ関数に渡します。

`main()`関数にて:

- 型が `String` の `result` 変数を作成します。
- `client` インスタンスに対して `run` スコープ関数を使用します。
- `run` スコープ関数内に一時的なスコープを作成するため、`client` インスタンスのプロパティや関数にアクセスする際に、明示的に `client` を参照する必要がなくなります。
- `connect()`、`authenticate()`、および `getData()` 関数を呼び出すラムダ式を、`run` スコープ関数に渡します。
- 結果を `result` 変数に代入します。

これで、返された結果をコード内でさらに活用できるようになりました。

Also

`also` スコープ関数を使用すると、オブジェクトに対して追加の処理（ログ出力など）を実行した後、そのオブジェクトを返して、コード内で引き続き使用することができます。

次の例を考えてみましょう：

```
fun main() {
    val medals: List<String> = listOf("Gold", "Silver", "Bronze")
    val reversedLongUpperCaseMedals: List<String> =
        medals
            .map { it.uppercase() }
            .filter { it.length > 4 }
            .reversed()
    println(reversedLongUpperCaseMedals)
    // [BRONZE, SILVER]
}
```

{kotlin-runnable="true" id="kotlin-tour-scope-function-also-before"}

この例では:

- 文字列のリストを含む `medals` 変数を作成します。
- `List<String>` 型の `reversedLongUpperCaseMedals` 変数を作成します。
- `medals` 変数に対して `.map()` 拡張関数を適用します。
- `.map()` 関数にラムダ式を渡します。このラムダ式は、`it` キーワードを介して `medals` を参照し、それに対して `.uppercase()` 拡張関数を呼び出します。
- `medals` 変数に対して `.filter()` 拡張関数を適用します。
- `.filter()` 関数に、`it` キーワードを介して `medals` を参照し、リスト内の項目が 4 文字以上であるかどうかを確認するラムダ式を述語として渡します。
- `medals` 変数に対して `.reversed()` 拡張関数を適用します。
- 結果を `reversedLongUpperCaseMedals` 変数に代入します。
- `reversedLongUpperCaseMedals` 変数に含まれるリストを出力します。

関数の呼び出しの間にログを出力するようにすると、`medals` 変数がどのように変化しているかを確認するのに役立つでしょう。その点で、`also` 関数が役立ちます：

```
fun main() {
    val medals: List<String> = listOf("Gold", "Silver", "Bronze")
    val reversedLongUppercaseMedals: List<String> =
        medals
            .map { it.uppercase() }
            .also { println(it) }
            // [GOLD, SILVER, BRONZE]
            .filter { it.length > 4 }
            .also { println(it) }
            // [SILVER, BRONZE]
            .reversed()
    println(reversedLongUppercaseMedals)
    // [BRONZE, SILVER]
}
```

{kotlin-runnable="true" id="kotlin-tour-scope-function-also-after"}

それでは、例を見てみましょう：

- `medals` 変数に対して `also` スコープ関数を使用します。
- `also` スコープ関数内に一時的なスコープを作成するため、関数の引数として `medals` 変数を使用する際に、明示的にその変数を参照する必要がなくなります。
- `it` キーワードを介して、`medals` 変数を関数引数として `println()` 関数を呼び出すラムダ式を、`also` スコープ関数に渡します。

`also` 関数はオブジェクトを返すため、ログ出力だけでなく、デバッグや複数の操作の連鎖、さらにはコードのメインフローに影響を与えないその他の副作用を伴う操作を行う際にも役立ちます。

With

他のスコープ関数とは異なり、`with` は拡張関数ではないため、構文が異なります。レシーバーオブジェクトを引数として `with` に渡します。

オブジェクトに対して複数の関数を呼び出したい場合は、`with` スコープ関数を使用します。

次の例を考えてみましょう：

```
class Canvas {
    fun rect(x: Int, y: Int, w: Int, h: Int): Unit = println("$x, $y, $w, $h")
    fun circ(x: Int, y: Int, rad: Int): Unit = println("$x, $y, $rad")
    fun text(x: Int, y: Int, str: String): Unit = println("$x, $y, $str")
}

fun main() {
    val mainMonitorPrimaryBufferBackedCanvas = Canvas()

    mainMonitorPrimaryBufferBackedCanvas.text(10, 10, "Foo")
}
```

```

mainMonitorPrimaryBufferBackedCanvas.rect(20, 30, 100, 50)
mainMonitorPrimaryBufferBackedCanvas.circ(40, 60, 25)
mainMonitorPrimaryBufferBackedCanvas.text(15, 45, "Hello")
mainMonitorPrimaryBufferBackedCanvas.rect(70, 80, 150, 100)
mainMonitorPrimaryBufferBackedCanvas.circ(90, 110, 40)
mainMonitorPrimaryBufferBackedCanvas.text(35, 55, "World")
mainMonitorPrimaryBufferBackedCanvas.rect(120, 140, 200, 75)
mainMonitorPrimaryBufferBackedCanvas.circ(160, 180, 55)
mainMonitorPrimaryBufferBackedCanvas.text(50, 70, "Kotlin")
}

```

```
{kotlin-runnable="true" id="kotlin-tour-scope-function-with-before"}
```

この例では、`rect()`、`circ()`、`text()` の 3 つのメンバ関数を持つ `Canvas` クラスを作成します。これらのメンバ関数はそれぞれ、指定された関数引数から構成された文を出力します。

この例では、`Canvas` クラスのインスタンスとして `mainMonitorPrimaryBufferBackedCanvas` を作成し、そのインスタンスに対して、異なる関数引数を用いて一連のメンバ関数を呼び出しています。

このコードは読みづらいことがお分かりいただけるでしょう。 `with` 関数を使用すると、コードが簡潔になります：

```

class Canvas {
    fun rect(x: Int, y: Int, w: Int, h: Int): Unit = println("$x, $y, $w, $h")
    fun circ(x: Int, y: Int, rad: Int): Unit = println("$x, $y, $rad")
    fun text(x: Int, y: Int, str: String): Unit = println("$x, $y, $str")
}

fun main() {
    //sampleStart
    val mainMonitorSecondaryBufferBackedCanvas = Canvas()
    with(mainMonitorSecondaryBufferBackedCanvas) {
        text(10, 10, "Foo")
        rect(20, 30, 100, 50)
        circ(40, 60, 25)
        text(15, 45, "Hello")
        rect(70, 80, 150, 100)
        circ(90, 110, 40)
        text(35, 55, "World")
        rect(120, 140, 200, 75)
        circ(160, 180, 55)
        text(50, 70, "Kotlin")
    }
    //sampleEnd
}

```

```
{kotlin-runnable="true" id="kotlin-tour-scope-function-with-after"}
```

この例では:

- `mainMonitorSecondaryBufferBackedCanvas` インスタンスをレシーバーとして、`with` スコープ関数を使用します。
- `with` スコープ関数内に一時的なスコープを作成するため、`mainMonitorSecondaryBufferBackedCanvas` インスタンスのメンバ関数を呼び出す際に、そのインスタンスを明示的に参照する必要がなくなります。
- 異なる関数引数を持つ一連のメンバ関数を呼び出すラムダ式を、`with` スコープ関数に渡します。

このコードがずっと読みやすくなったので、ミスを犯す可能性も低くなりました。

Use case overview

ユースケースの概要

このセクションでは、Kotlinで利用できるさまざまなスコープ関数と、コードをよりイディオム的なもの（よりKotlinらしい）にするための主な活用例について解説しました。この表を手引きとしてご利用ください。これらの関数がどのように動作するかを完全に理解していなくても、コード内でそれらを使用することは可能であるという点に留意してください。

機能	x へのアクセス	Return value	ユースケース
<code>let</code>	<code>it</code>	ラムダ式の結果	コード内でNULLチェックを行い、その後、返されたオブジェクトに対してさらなる処理を実行する。
<code>apply</code>	<code>this</code>	x	オブジェクトは作成時に初期化する。
<code>run</code>	<code>this</code>	ラムダ式の結果	オブジェクトを生成時に初期化し、 かつ 結果を計算する。
<code>also</code>	<code>it</code>	x	オブジェクトを返す前に、追加の処理をすべて完了させる。
<code>with</code>	<code>this</code>	ラムダ式の結果	オブジェクトに対して複数の関数を呼び出す。

スコープ関数に関する詳細については、[スコープ関数](#)を参照してください。

演習 {completion-point="true"}

課題 1 {initial-collapse-state="collapsed" collapsible="true" id="scope-functions-exercise-1"}

`.getPriceInEuros()` 関数を、安全な呼び出し演算子 `?.` と `let` のスコープ機能を使用した単一式関数として書き直してください。

`?.` といったセーフコール演算子を使用して、`getProductInfo()` 関数から `priceInDollars` プロパティに安全にアクセスします。その後、`let` スコープ関数を使用して、`priceInDollars` の値をユーロに変換します。

|---|---|

```
data class ProductInfo(val priceInDollars: Double?)
```

```

class Product {
    fun getProductInfo(): ProductInfo? {
        return ProductInfo(100.0)
    }
}

// Rewrite this function
fun Product.getPriceInEuros(): Double? {
    val info = getProductInfo()
    if (info == null) return null
    val price = info.priceInDollars
    if (price == null) return null
    return convertToEuros(price)
}

fun convertToEuros(dollars: Double): Double {
    return dollars * 0.85
}

fun main() {
    val product = Product()
    val priceInEuros = product.getPriceInEuros()

    if (priceInEuros != null) {
        println("Price in Euros: €$priceInEuros")
        // Price in Euros: €85.0
    } else {
        println("Price information is not available.")
    }
}

```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-scope-functions-exercise-1"}

|---|---|

```

data class ProductInfo(val priceInDollars: Double?)

class Product {
    fun getProductInfo(): ProductInfo? {
        return ProductInfo(100.0)
    }
}

fun Product.getPriceInEuros() = getProductInfo()?.priceInDollars?.let {
    convertToEuros(it) }

fun convertToEuros(dollars: Double): Double {
    return dollars * 0.85
}

fun main() {

```

```

val product = Product()
val priceInEuros = product.getPriceInEuros()

if (priceInEuros != null) {
    println("Price in Euros: €$priceInEuros")
    // Price in Euros: €85.0
} else {
    println("Price information is not available.")
}
}

```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-scope-functions-solution-1"}

課題 2 {initial-collapse-state="collapsed" collapsible="true" id="scope-functions-exercise-2"}

ユーザーのメールアドレスを更新する `updateEmail()` 関数があります。 `apply` スコープ関数を使用してメールアドレスを更新し、続いて `also` スコープ関数を使用して「ID: `{it.id}` のユーザーのメールアドレスを更新中」というログメッセージを出力します。

|---|---|

```

data class User(val id: Int, var email: String)

fun updateEmail(user: User, newEmail: String): User = // Write your code here

fun main() {
    val user = User(1, "old_email@example.com")
    val updatedUser = updateEmail(user, "new_email@example.com")
    // Updating email for user with ID: 1

    println("Updated User: $updatedUser")
    // Updated User: User(id=1, email=new_email@example.com)
}

```

{validate="false" kotlin-runnable="true" kotlin-min-compiler-version="1.3" id="kotlin-tour-scope-functions-exercise-2"}

|---|---|

```

data class User(val id: Int, var email: String)

fun updateEmail(user: User, newEmail: String): User = user.apply {
    this.email = newEmail
}.also { println("Updating email for user with ID: ${it.id}") }

fun main() {
    val user = User(1, "old_email@example.com")
    val updatedUser = updateEmail(user, "new_email@example.com")
}

```

```
// Updating email for user with ID: 1

println("Updated User: $updatedUser")
// Updated User: User(id=1, email=new_email@example.com)
}
```

{initial-collapse-state="collapsed" collapsible="true" collapsed-title="Example solution" id="kotlin-tour-scope-functions-solution-2"}

- [Previous step](#)
- [Next step](#)